Grado Superior

Programación

T5. Programación Orientada a Objetos

Curso 2025/26

Profesor

Raúl Murillo Gallego

«El momento en que quieres dejarlo es justo el momento en que tienes que seguir avanzando» —Anónimo

Presentación

5.1. Introducción
5.2. Auge de la programación orientada a objetos
5.3. Características de la programación orientada a objetos
5.3.1. Clases
5.3.2. Atributos
5.3.3. Métodos
5.3.4. Constructores
5.3.5. Getter y Setter
5.3.6. Garbage Collector
5.3.7. Contexto de clase. Modificador STATIC

T5. Programación Orientada a Objetos

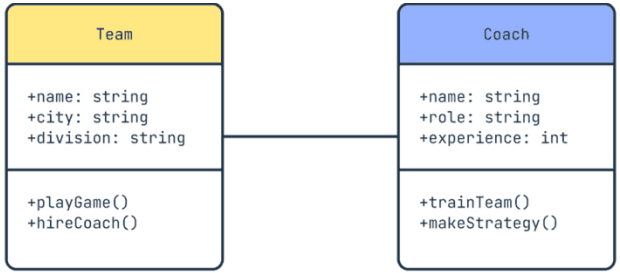
5.1. Introducción
5.2. Auge de la programación orientada a objetos
5.3. Características de la programación orientada a objetos
5.3.1. Clases
5.3.2. Atributos
5.3.3. Métodos
5.3.4. Constructores
5.3.5. Getter y Setter
5.3.6. Garbage Collector
5.3.7. Contexto de clase. Modificador STATIC

5.1. Introducción

Introducción

Al igual que la programación estructurada, la **programación orientada a objetos** (POO) es un tipo de programación que desciende del paradigma de programación imperativa. Tiene sus orígenes en el lenguaje de programación Simula 67, que fue el primero en introducir los conceptos de *objeto* y *clase* en el año 1962.

En este tema, se introducirá el paradigma de programación orientada a objetos, describiendo los distintos elementos que lo conforman e introduciendo pequeños conceptos de diseño mediante el **Lenguaje Unificado del Modelado** (UML) que facilitan la comprensión a la vez que permiten ilustrar ejemplos.



T5. Programación Orientada a Objetos

5.1. Introducción
5.2. Auge de la programación orientada a objetos
5.3. Características de la programación orientada a objetos
5.3.1. Clases
5.3.2. Atributos
5.3.3. Métodos
5.3.4. Constructores
5.3.5. Getter y Setter
5.3.6. Garbage Collector
5.3.7. Contexto de clase. Modificador STATIC

5.2. Auge de la programación orientada a objetos

Auge de la Programación Orientada a Objetos

La POO se ha convertido en el paradigma de programación más empleado desde la década de los ochenta y eso se debe a las muchas e importantes ventajas que aporta con respecto a otros paradigmas de programación

- Permite enfocar los problemas a resolver de una forma mucho más **intuitiva** de lo que lo hacen otros paradigmas. En este paradigma se persigue trabajar con un modelo más cercano al mundo real en el que los diferentes **objetos** interaccionan entre sí.

- La POO permite generar programas modulares y fácilmente **escalables**.

- Su capacidad para **reutilizar código** sin necesidad reescribirlo nuevamente facilitó mucho la tarea a los programadores.

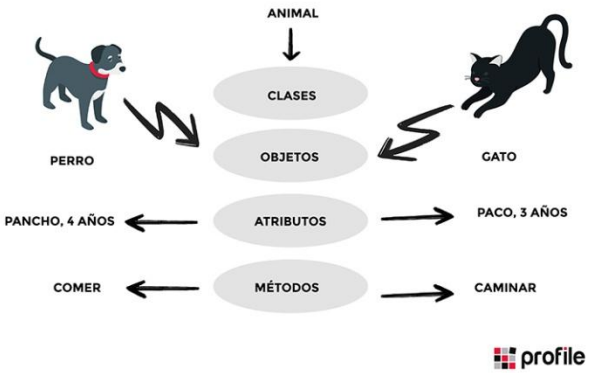
T5. Programación Orientada a Objetos

5.1. Introducción
5.2. Auge de la programación orientada a objetos
5.3. Características de la programación orientada a objetos
5.3.1. Clases
5.3.2. Atributos
5.3.3. Métodos
5.3.4. Constructores
5.3.5. Getter y Setter
5.3.6. Garbage Collector
5.3.7. Contexto de clase. Modificador STATIC

5.3. Características de la programación orientada a objetos

Características de la POO

En la programación orientada a objetos, los programas se estructuran alrededor de «**objetos**», que son instancias de clases que encapsulan datos y métodos. Este enfoque se basa en conceptos como la **abstracción**, la **encapsulación**, la **herencia** y el **polimorfismo**. Lenguajes como Java, Python y C# son conocidos por seguir este paradigma.



Características de la POO

“Imaginad que estáis diseñando un videojuego. Tenéis personajes, enemigos, armas, escenarios… ¿Cómo organizáis todo eso en código sin que sea un caos?

Aquí entra la Programación Orientada a Objetos, o POO. Es como construir con piezas de LEGO: cada pieza tiene una forma, una función, y se puede combinar con otras para crear algo más grande.

En POO, esas piezas son los objetos. Cada objeto tiene:
Propiedades (como el color, tamaño, vida, nombre…)
Acciones (como moverse, atacar, hablar…)

Y todos los objetos se crean a partir de clases, que son como moldes o planos.
Por ejemplo, si creamos una clase Coche, podemos tener objetos como MiCoche, TuCoche, SuCoche, cada uno con sus propios datos y comportamientos.

Clase

La **clase** es el elemento básico de programación en paradigma orientado a objetos. Se puede definir como:

- Una clase es un molde o plantilla a partir de la cual se crean objetos.
- Define **atributos** (también llamados propiedades o variables de instancia) y **métodos** (también llamados comportamientos o funciones) que los objetos creados a partir de la clase tendrán.
- No puede ser utilizado directamente (solamente mediante instancias al objeto)
- Ejemplos (Persona, Animal, Coche, Producto).

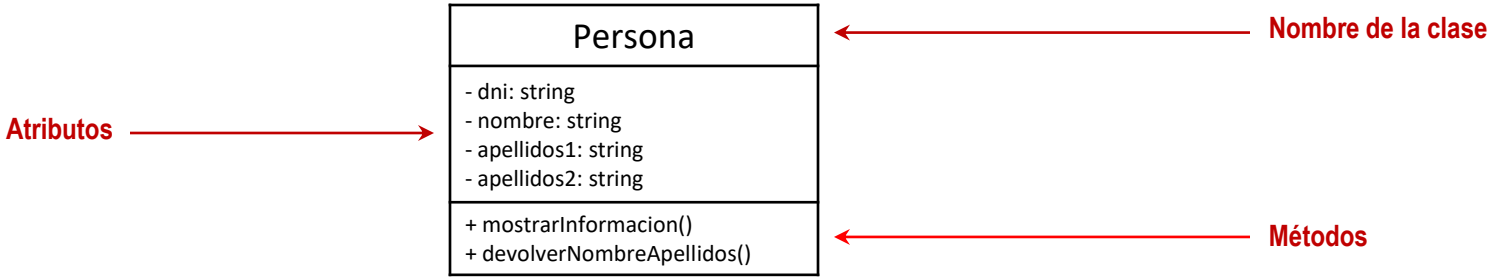


Fig. Representación de una clase

Clase (atributos)

Un **atributo de una clase** es una **propiedad o característica** que representa los datos o el estado asociado a los objetos de esa clase. Los atributos definen qué información puede contener un objeto y suelen ser representados por variables dentro de la clase.

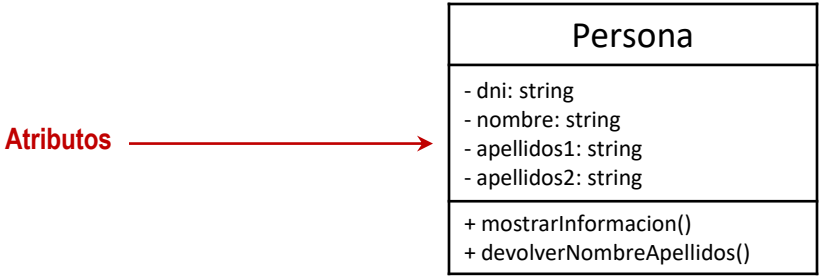


Fig. Representación de una clase

```
public class Persona
{
        private String dni;
        private String nombre;
        private String apellidos1;
        private String apellidos2;
}
```

Clase (atributos)

Características principales de un atributo:

- **Pertenencia**: Los atributos están definidos dentro de una clase y describen el estado o la información que cada instancia (objeto) de esa clase puede tener.
- **Tipo**: Cada atributo tiene un **tipo de dato**, como enteros (`int`), cadenas de texto (`String`), booleanos (`boolean`), entre otros.
- **Visibilidad o Modificadores de acceso**: Los atributos pueden tener diferentes niveles de acceso:
  - *public*: El atributo es accesible desde cualquier parte del código.
  - *private*: El atributo solo es accesible dentro de la propia clase.
  - *protected*: El atributo es accesible dentro de la clase, subclases, y dentro del mismo paquete.
- **Estado**: Los atributos definen el **estado** interno del objeto. Por ejemplo, si tienes una clase Coche, los atributos podrían ser color, modelo, y velocidad.

Clase (atributos)

Ejemplos de atributos de una clase:

```
public class Coche
{
      private String color;
      private String modelo;
      private int velocidad;
}
```

```
public class Persona
{
      private String dni;
      private String nombre;
      private String apellidos;
}
```

```
public class Factura
{
      private int numeroFactura;
      private String descripcion;
      private double importeFactura;
}
```

Clase (métodos)

Un **método** es una función definida dentro de una clase que describe **un comportamiento o una acción** que los objetos de esa clase pueden realizar. Los métodos permiten a los objetos interactuar con sus atributos o con otros objetos, y definen las operaciones que se pueden realizar sobre esos datos.

```
public class Persona
{
        public void mostrarInformacion(){
        }
}
```

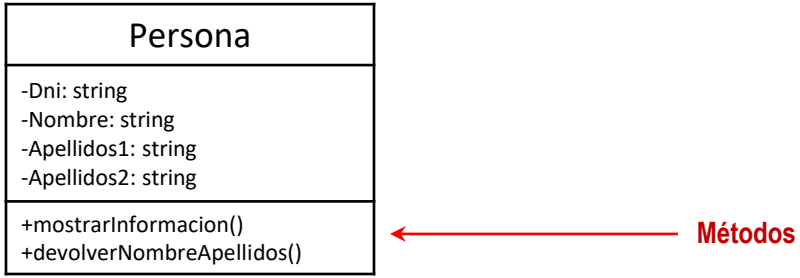


Fig. Representación de una clase

Clase (métodos)

Características principales de un método:

- **Pertenencia a una clase:** Un método está asociado a una clase y define un comportamiento o una acción que los objetos de esa clase pueden ejecutar. Suelen contener un verbo en infinitivo (mostrar, obtener, devolver, calcular…)
- **Comportamiento o funcionalidad**: Los métodos implementan lógica que permite manipular o interactuar con los atributos del objeto y con otros objetos, como en el caso de cálculos, acciones o transformaciones.
- **Visibilidad o Modificadores de acceso**: Los métodos pueden tener modificadores de acceso como public, private o protected, que determinan desde dónde pueden ser invocados:
    - *public*: Puede ser llamado desde cualquier parte del programa.
    - *private*: Solo puede ser llamado dentro de la propia clase.
    - *protected*: Puede ser accedido desde la propia clase, sus subclases, y desde el mismo paquete.
- Pueden tener opcionalmente **valores de entrada** (parámetros) y **valores de salida** (valores de retorno)
- Pueden existir **procedimientos** (no retornan ninguna valor) y **funciones** (retornan un valor)

Clase (métodos)

Ejemplos de métodos de una clase:

```
public class Coche
{
      private int velocidad;

      // Método que devuelve la velocidad actual del coche
      public int obtenerVelocidad()
      {
             return velocidad;
      }
}
```

```
public class Persona
{
      private String dni;
      private String nombre;
      private String apellidos;

      // Método que devuelve información de una cadena de texto
      public void mostrarInformacion()
      {
        System.out.println("Aquí muestro información");
      }
      // Método que devuelve el DNI
      public void devolverNombreApellidos(boolean tieneApellidos)
      {
          if (tieneApellidos)
          {
             System.out.println("Esta persona tiene nombre y apellidos");
          }
        else
        {
            System.out.println("Esta persona solo tiene nombre");
        }
    }
}
```

Clase (constructores)

Un **constructor** es un **método especial** que se utiliza para **inicializar** un objeto cuando se crea una nueva instancia de una clase. Su función principal es asignar valores iniciales a los atributos del objeto o realizar cualquier configuración necesaria. A diferencia de otros métodos, el constructor se ejecuta automáticamente al crear un objeto, y no tiene un valor de retorno explícito.

```
public class Persona
{
        public Persona(){
        }
}
```

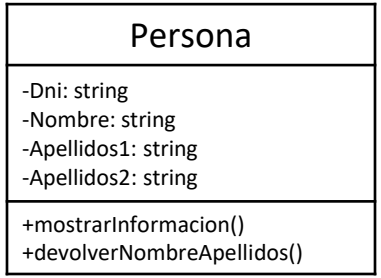


Fig. Representación de una clase

Clase (constructores)

Características principales de un constructor:

- **Nombre igual al de la clase:** El nombre del constructor debe coincidir **exactamente** con el nombre de la clase en la que está definido.
- **Sin tipo de retorno:** A diferencia de los métodos normales, un constructor no tiene tipo de retorno, ni siquiera void.
- **Inicialización de atributos:** Los constructores permiten inicializar los atributos de un objeto, asignando valores predeterminados o valores pasados como parámetros.
- **Sobrecarga de constructores**: Se pueden definir múltiples constructores en una clase, siempre que tengan diferentes números o tipos de parámetros. Esto se conoce como sobrecarga de constructores y permite crear objetos con diferentes configuraciones iniciales.
- **Constructor por defecto:** Si no se define un constructor en una clase, el lenguaje proporciona un constructor por defecto (sin parámetros) que inicializa los atributos del objeto con valores predeterminados.

Clase (constructores)

Ejemplos de constructores de una clase:

```
public class Coche
{
     private String color;
     private int velocidad;

      public Coche()
      {
      }

     public Coche(String color, int velocidad)
     {
          this.color = color;
          this.velocidad = velocidad;
       }
}
```

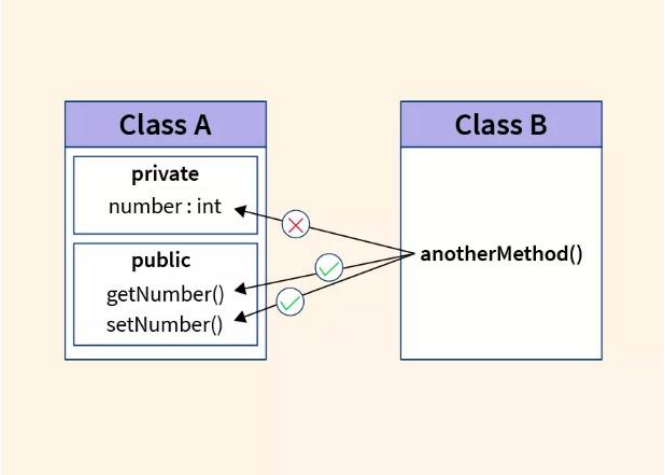
```
public class Persona
{
    private String dni;
    private String nombre;
    private String apellidos;

    public Persona() {
    }

    public Persona(String dni, String nombre, String
apellidos) {
        this.dni = dni;
        this.nombre = nombre;
        this.apellidos = apellidos;
    }
}
```

Clase (getter y setter)

Los **getters** y **setters** son métodos especiales utilizados para acceder y modificar los atributos privados de un objeto. La **encapsulación**, un principio clave de POO, sugiere que los atributos de una clase deben ser privados, es decir, no accesibles directamente desde fuera de la clase. Para interactuar con estos atributos, se utilizan los **getters** y **setters**, que permiten un acceso controlado y seguro a los datos del objeto.



Clase (getter)

Un **getter** (también conocido como **accesor**) es un método que se utiliza para **obtener** o **consultar** el valor de un atributo privado de un objeto.

- Generalmente, el nombre del método comienza con la palabra `get` seguida del nombre del atributo, y su propósito es devolver el valor del atributo al que está asociado.
- Los **getters** suelen ser métodos públicos, lo que permite que otras clases accedan a los valores de los atributos, sin modificar su estado directamente.

```
public class Persona
{
    private String nombre;

    // Getter para el atributo 'nombre'
    public String getNombre() {
           return nombre;
     }
}
```

Clase (setter)

Un setter (también conocido como **mutador**) es un método que se utiliza para **modificar** o **establecer** el valor de un atributo privado de un objeto.

- El nombre del método suele comenzar con `set` seguido del nombre del atributo, y su propósito es asignar un nuevo valor al atributo.
- Los setters proporcionan una manera controlada de modificar los atributos de un objeto, lo que puede incluir lógica adicional, como validación, antes de permitir que el valor se cambie.

```
public class Persona
{
    private String nombre;

    // Setter para el atributo 'nombre'
    public void setNombre(String nombre) {
          this.nombre = nombre;
    }
}
```

Clase (objetos)

Un objeto es una instancia de una clase determinada. Dicho de otra manera, se puede decir que un objeto es el resultado de crear en tiempo de ejecución un conjunto de datos que se almacena en memoria y que tiene una estructura como la que se describe en su clase.

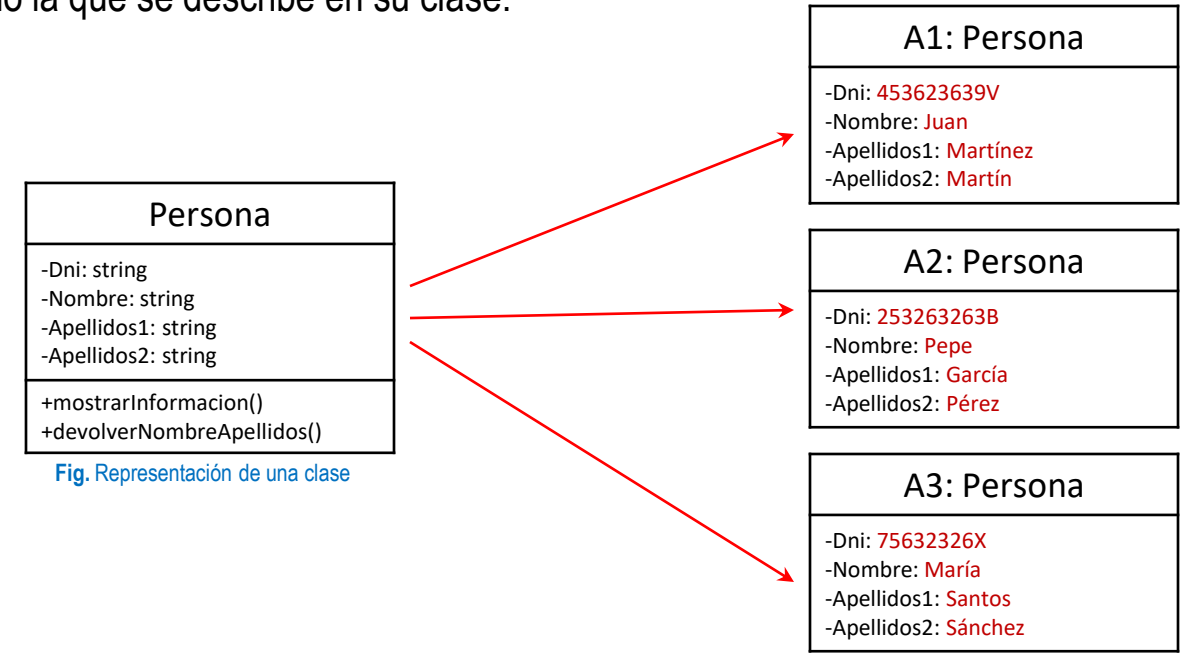


Fig. Representación de una clase

Clase (objetos)

Características principales de un objeto:

- **Instancia de una clase:** Un objeto se crea a partir de una **clase**, que actúa como un molde o plantilla que define las características y comportamientos que tendrá el objeto. Cada objeto es una **instancia** única de esa clase.
- **Estado:** El estado de un objeto está definido por sus **atributos** o **propiedades**. Cada atributo tiene un valor que describe la condición actual del objeto. Estos valores pueden cambiar a lo largo del tiempo.
- **Comportamiento:** El comportamiento de un objeto está definido por los **métodos** de la clase. Los métodos son las acciones que el objeto puede realizar o las operaciones que pueden afectar su estado o interactuar con otros objetos.
- **Identidad:** Aunque dos objetos pueden ser de la misma clase y tener el mismo estado, cada uno tiene una **identidad única** que los distingue entre sí, lo que significa que ocupan diferentes posiciones en la memoria.

Clase (objetos)

Ejemplo:

```
public class Main {
    public static void main (String[] args){
        Persona objPersona = new Persona();

        objPersona.setNombre("Pepe");
        objPersona.setApellidos("Martinez");
        objPersona.setDni("44455625R");

        objPersona.mostrarInfo();
        String resultadoFuncion = objPersona.devolverNombreApellidos();

        System.out.println("Resultado de la llamada a la funcion es " + resultadoFuncion);
    }
}
```

```
private String Dni;
    private String nombre;
    private String apellidos;
    public Persona() {
    }
    public Persona(String dni, String nombre, String apellidos) {
        this.Dni = dni;
        this.nombre = nombre;
        this.apellidos = apellidos;
    }
    public String getDni() {
        return Dni;
    }
    public void setDni(String Dni) {
        this.Dni = Dni;
    }
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public String getApellidos() {
        return apellidos;
    }
    public void setApellidos(String apellidos) {
        this.apellidos = apellidos;
    }
    public void mostrarInfo(){
        System.out.println("El DNI es" + this.Dni);
    }
    public String devolverNombreApellidos (){
        String cadena = this.nombre + " " + this.apellidos;
        return cadena;
    }
```

Clase (objetos)

A1: Persona

-Dni: 453623639V
-Nombre: Juan
-Apellidos1: Martínez
-Apellidos2: Martín

A2: Persona

-Dni: 253263263B
-Nombre: Pepe
-Apellidos1: García
-Apellidos2: Pérez

A3: Persona

-Dni: 75632326X
-Nombre: María
-Apellidos1: Santos
-Apellidos2: Sánchez

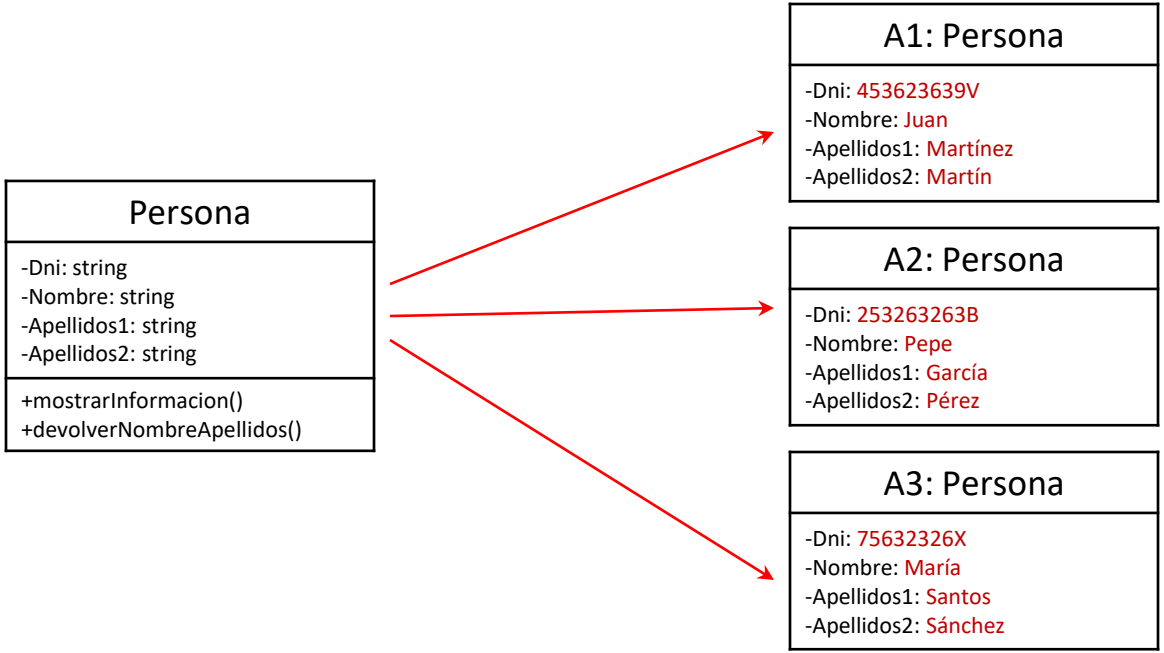
Objetos de la clase Persona

Garbage Collector

En Java, el Garbage Collector (GC) es responsable de liberar automáticamente la memoria de objetos que ya no se utilizan. No necesitamos invocarlo explícitamente en la mayoría de los casos, ya que Java se encarga de administrar la memoria automáticamente.

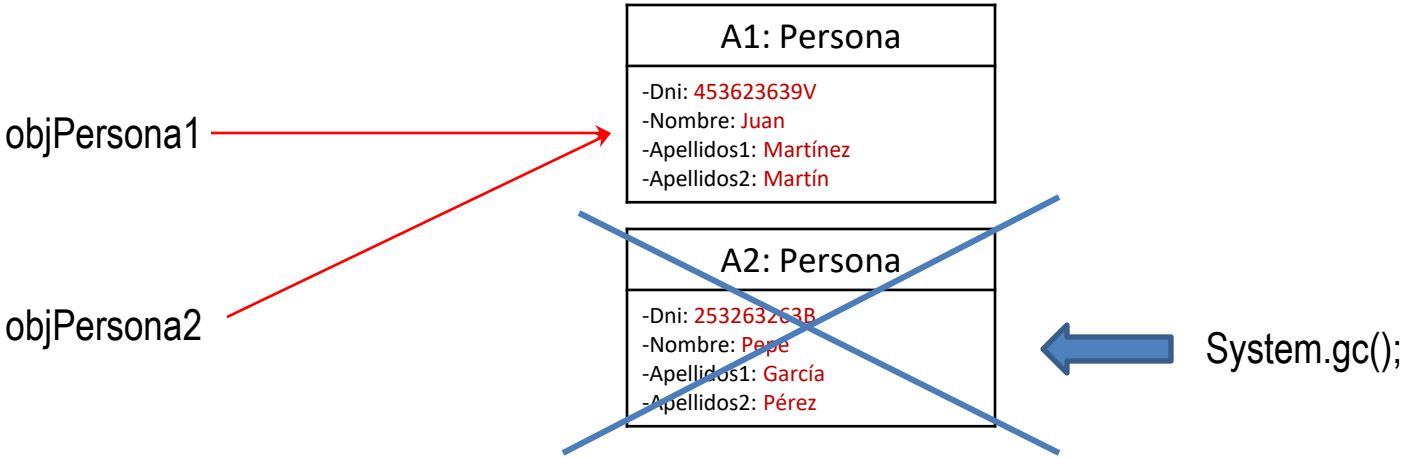
Garbage Collector

Ejemplo:



Garbage Collector

```
public class Main {
    public static void main(String[] args) {
        Persona objPersona1("453623639V", "Juan","Martínez","Martin");
        Persona objPersona2("785658696E", "Pepe","Herrero","Pelayo");
        objPersona1 = objPersona2
    }
}
```



Contexto de clase. Modificador STATIC

Todos los conceptos tratados hasta ahora en este tema se han desarrollado en contexto de **objeto**. Dicho contexto implica la instanciación de objetos diferentes, cada uno con su propio “juego” de atributos de objeto, y cada uno, una ocupación de memoria. Existe también la posibilidad de actuar en un contexto de **clase**, es decir, prescindiendo de la existencia de objetos, aunque no es incompatible actuar en contexto de clase existiendo objetos.

```
public static int devolverNumeroMayor(int numeroA, int numeroB) {
        if numeroA > numeroB{
                return numeroA ;
        }
        else {
                return numeroB;
        }
}
```

Invocar:

```
Alumno.devolverNumeroMayor(5,7)
```

Contexto de clase. Modificador `STATIC`

¿Cuándo usar `static` en POO?

- **Usa `static` para métodos y atributos que no dependan de instancias individuales** de una clase.
- **Para métodos de utilidad o helpers**, que suelen realizar operaciones que no están ligadas a los datos de una instancia (ej. matemáticas, conversiones, etc.).
- **Para atributos o constantes** de la clase que deben compartirse entre todas las instancias (ej. contadores, configuraciones globales, etc.).

Ventajas y Consideraciones

- **Eficiencia y uso compartido**: Los métodos y atributos `static` se almacenan en memoria una sola vez, independientemente del número de instancias, lo cual puede ahorrar memoria.
- **Dependencia del estado**: Evita hacer `static` aquellos métodos o variables que dependan del estado de la instancia, ya que perderían el contexto individual y podrían causar efectos secundarios inesperados.

Grado Superior

Programación

T5. Programación Orientada a Objetos

Curso 2025/26

Profesor

Raúl Murillo Gallego

«El momento en que quieres dejarlo es justo el momento en que tienes que seguir avanzando» —Anónimo